Q7NathanMehta

September 27, 2024

1 Q7

7.1 data loading and peek

[1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import plotly.express as px
import plotly.io as pio
```

[2]:
```
pitchdata = pd.read_csv("AnalyticsQuestionnairePitchData.csv")
```

[3]:
```
pitchdata.head()
```

[3]:
```
   PitchId  GamePk PitcherHand        PitchCall PitchType  BatterId  \
0      231       1           L    called_strike        CH        19
1      384       1           L             ball        CU        19
2      616       1           L        field_out        SL        19
3      503       1           L             ball        SI        24
4      322       1           L  swinging_strike        CU        24

   PitcherId BatterSide  Inning  IsTop  …  TrajectoryPolynomialZ2  \
0         11          R       1      1  …              -14.281734
1         11          R       1      1  …              -19.321169
2         11          R       1      1  …              -13.228232
3         11          L       1      1  …              -12.885942
4         11          L       1      1  …              -18.384531

   TrajectoryX0  TrajectoryX1  TrajectoryX2  TrajectoryV0  TrajectoryV1  \
0      1.629367     50.002632      5.681500     -5.673969   -120.768692
1      1.694219     50.001293      5.523830     -6.150516   -115.130562
2      1.853305     50.000290      5.531102     -5.501207   -127.250221
3      1.891533     50.001602      5.475145     -4.826164   -133.365570
4      1.784340     50.002003      5.493305     -5.491218   -116.880058

   TrajectoryV2  TrajectoryA0  TrajectoryA1  TrajectoryA2
0     -3.060989      8.499877     21.381056    -28.563467
```

```
1     -1.678751     -2.671943     17.934855    -38.642338
2     -1.467974      0.618823     21.358961    -26.456465
3     -2.412576     14.973106     24.863359    -25.771883
4     -2.260371      0.099588     18.352825    -36.769062

[5 rows x 67 columns]
```

1.1 7.2 Performance KPIs by pitcher

1.1.1 KPIs of interest: WHIP, On Base %, SLG, OPS, K/9, K-BB%

1.2 WHIP:

using innings appeared in instead of innings pitched

[5]:
```
#finding number of innings pitched total for each pitcher
innings_pitched = pitchdata.groupby(['PitcherId', 'GamePk', 'Inning']).size().
 ↪to_frame().reset_index()
innings_pitched2 = innings_pitched.groupby(['PitcherId']).size().to_frame().
 ↪reset_index()
innings_pitched_final = innings_pitched2.rename({0:'IP'}, axis = 1)
```

[6]:
```
#totals of each pitchcall for each pitcher
pitch_calls_by_pitcher = pitchdata.groupby(['PitcherId', 'PitchCall']).size().
 ↪to_frame().reset_index().rename({0:'Count'}, axis = 1)
```

[7]:
```
#join IP with pitchcalls totals for each pitcher
pitchcalls_IP = pitch_calls_by_pitcher.merge(innings_pitched_final, how='left',␣
 ↪on='PitcherId')
```

[8]:
```
# Search pitchcalls for each pitcher for walks and hits. divide by innings␣
 ↪appeared in for each
walkorhit = ['walk','single','double','triple','home_run']
whipdf = []
for pitcher in pitchcalls_IP['PitcherId'].unique().tolist():
    p_data = pitchcalls_IP[pitchcalls_IP['PitcherId'] == pitcher]
    p_wph = p_data[p_data['PitchCall'].isin(walkorhit)]['Count'].values.sum()
    whip = p_wph / p_data['IP'].values[0]
    whipdf.append([pitcher, whip, p_data['IP'].values[0]])
whipdf = pd.DataFrame(whipdf, columns=['PitcherId', 'WHIP', 'IP'])
whipdf
```

[8]:

	PitcherId	WHIP	IP
0	1	0.916667	12
1	2	0.000000	1
2	3	0.500000	2
3	4	1.000000	1
4	5	2.000000	1

```
5           6  0.000000   2
6           7  1.000000   6
7           8  1.000000   1
8           9  1.000000   1
9          10  3.000000   1
10         11  1.285714   7
11         12  3.000000   1
12         13  1.000000   1
13         14  2.000000   1
14         15  0.000000   1
15         16  1.000000   1
16         17  2.000000   1
```

- Across both games, pitcher 1 pitched almost double the number of innings than the next most productive (7 innings by pitcher 11)
- With the workload, pitcher 1 impressively allowed less than 1 walk or hit per inning, indicating control of the basepaths and limiting of damage. Pitcher 11 allowed 1.2 WHIP, allowing more traffic.
- Pitchers 10 and 12 were susceptible to allow bases, 3 WHIP each in their single inning of work

1.3 OBP:

[9]:
```
#group by pitcher, game, at bat, pitch call
pitcher_atbats_calls = pitchdata.groupby(['PitcherId', 'GamePk', 'AtBatNumber',␣
↪'PitchCall']).size().to_frame().reset_index()
pitcher_atbats_calls2 = pitcher_atbats_calls.rename({0:'Count'}, axis = 1)
```

[10]:
```
#append each pitcher's OBP allowed to a dataframe with total At Bats
obp_df = []
for pitcher in pitcher_atbats_calls2['PitcherId'].unique().tolist():
    p_data = pitcher_atbats_calls2[pitcher_atbats_calls2['PitcherId'] ==␣
↪pitcher]
    p_onbase = p_data[p_data['PitchCall'].isin(walkorhit)]['Count'].values.sum()
    ABs = len(p_data['AtBatNumber'].unique())
    obp = (p_onbase) / (ABs)
    obp_df.append([pitcher, obp, ABs])
obp_df = pd.DataFrame(obp_df, columns=['PitcherId', 'OBP-allowed', 'At-Bats'])
obp_df
```

[10]:
```
    PitcherId  OBP-allowed  At-Bats
0           1     0.282051       39
1           2     0.000000        3
2           3     0.200000        5
3           4     0.250000        4
4           5     0.400000        5
5           6     0.000000        6
```

```
6           7     0.230769       26
7           8     0.333333        3
8           9     0.250000        4
9          10     0.500000        6
10         11     0.346154       26
11         12     0.500000        6
12         13     0.333333        3
13         14     0.400000        5
14         15     0.000000        3
15         16     0.500000        2
16         17     0.400000        5
```

- Looking at On-base percentage allowed by pitcher, **pitcher 1** has allowed a baserunner about **28%** of ~40 at-bats faced over the 2 games. This is proficient, but perhaps **pitcher 7** was more efficient in putting out batters, allowing a **23%** on-base percentage for 26 batters.
- Corroborating the WHIP analysis, **pitcher 11** hasn't had quite had the put-out ability that pitchers 1 and 7 display, allowing **35%** OBP in 26 innings.
- In accordance with WHIP, **pitchers 10 and 12** allowed **50%** on-base while each facing 6 batters
- **Pitcher 6** was perfect, allowing no walks or hits in his 2 innings of relief

[12]:
```
walk_or_ball = ['walk', 'ball', 'wild_pitch']
strike_or_putout = ['called_strike','field_out','swinging_strike','foul',␣
↪'strikeout', 'foul_bunt', 'foul_tip', 'force_out', 'swinging_strike_blocked']
```

1.4 SLG:

[13]:
```
slg_df = []
for pitcher in pitcher_atbats_calls2['PitcherId'].unique().tolist():
    p_data = pitcher_atbats_calls2[pitcher_atbats_calls2['PitcherId'] ==␣
↪pitcher]
    p_singles = p_data[p_data['PitchCall'] == 'single']['Count'].values.sum()
    p_doubles = p_data[p_data['PitchCall'] == 'double']['Count'].values.sum()
    p_triples = p_data[p_data['PitchCall'] == 'triple']['Count'].values.sum()
    p_hrs = p_data[p_data['PitchCall'] == 'home_run']['Count'].values.sum()
    ABs = len(p_data['AtBatNumber'].unique())
    slg = (p_singles + (2*p_doubles) + (3*p_triples) + (4*p_hrs)) / (ABs)
    slg_df.append([pitcher, slg, ABs])
slg_df = pd.DataFrame(slg_df, columns=['PitcherId', 'SLG-allowed', 'At-Bats'])
slg_df
```

[13]:
```
    PitcherId  SLG-allowed  At-Bats
0           1     0.333333       39
1           2     0.000000        3
2           3     0.200000        5
```

3	4	0.000000	4
4	5	0.600000	5
5	6	0.000000	6
6	7	0.269231	26
7	8	0.000000	3
8	9	0.250000	4
9	10	0.333333	6
10	11	0.346154	26
11	12	0.500000	6
12	13	0.333333	3
13	14	0.800000	5
14	15	0.000000	3
15	16	0.500000	2
16	17	0.200000	5

- **Slugging % allowed** measures pitcher ability to limit extra-base hits and therefore runs. It can be interpreted as the number of bases given up per at-bat.
- **Pitcher 1** gave up about **1 base every 3 at-bat**, a decent mitigation of batting power. **pitcher 7** once again shows most prolific ability to stifle hitters, allowing ~ **1 base every 4 batters** for 26 batters. Once again, **pitcher 11** eats up innings with an adequate outing but allows .34 SLG, higher than pitchers 1 and 7.
- pitchers **5, 12, 14** were susceptible to being hit hard in their limited outings

1.5 OPS

[14]:
```
ops_df = slg_df.merge(obp_df, how='left', on=['PitcherId', "At-Bats"])
ops_df['OPS'] = ops_df['SLG-allowed'] + ops_df['OBP-allowed']
ops_df['WHIP'] = whipdf['WHIP']
```

[15]:
```
# join OBP and SLG data for OPS data sorted by at-bats faced
ops_df = ops_df.sort_values(by='At-Bats', ascending=False)
ops_df
```

[15]:

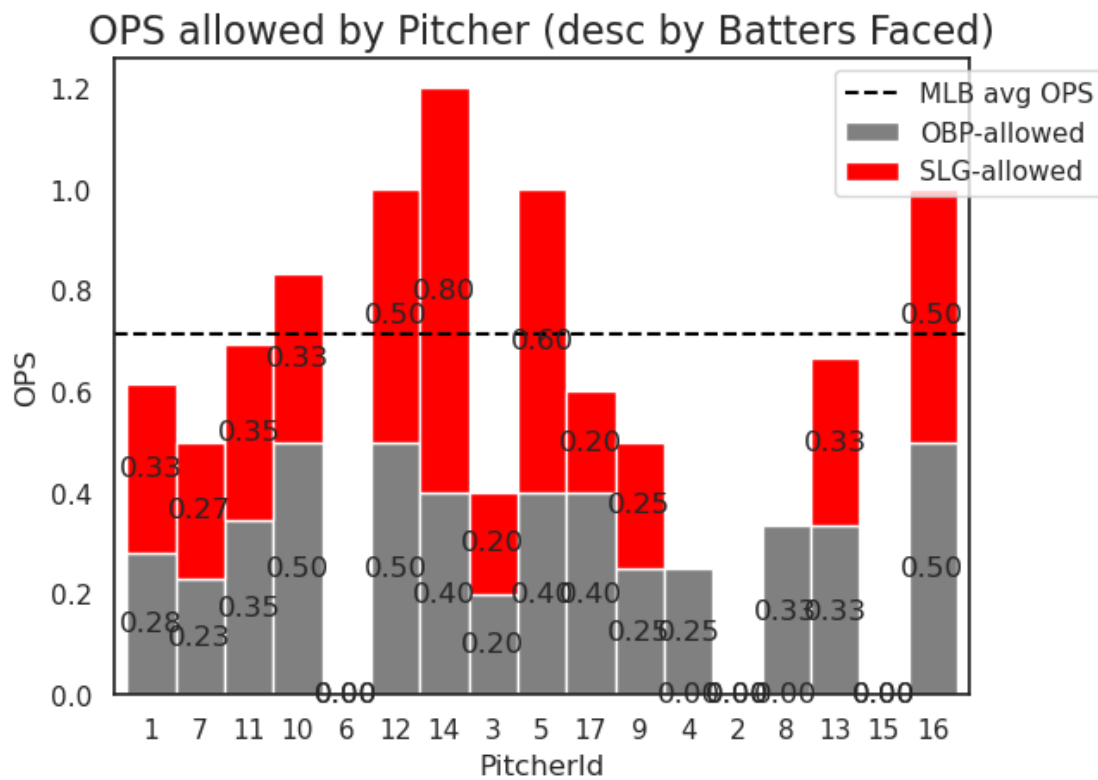
	PitcherId	SLG-allowed	At-Bats	OBP-allowed	OPS	WHIP
0	1	0.333333	39	0.282051	0.615385	0.916667
6	7	0.269231	26	0.230769	0.500000	1.000000
10	11	0.346154	26	0.346154	0.692308	1.285714
9	10	0.333333	6	0.500000	0.833333	3.000000
5	6	0.000000	6	0.000000	0.000000	0.000000
11	12	0.500000	6	0.500000	1.000000	3.000000
13	14	0.800000	5	0.400000	1.200000	2.000000
2	3	0.200000	5	0.200000	0.400000	0.500000
4	5	0.600000	5	0.400000	1.000000	2.000000
16	17	0.200000	5	0.400000	0.600000	2.000000
8	9	0.250000	4	0.250000	0.500000	1.000000
3	4	0.000000	4	0.250000	0.250000	1.000000
1	2	0.000000	3	0.000000	0.000000	0.000000

```
7            8     0.000000        3     0.333333  0.333333  1.000000
12          13     0.333333        3     0.333333  0.666667  1.000000
14          15     0.000000        3     0.000000  0.000000  0.000000
15          16     0.500000        2     0.500000  1.000000  1.000000
```

Plot OPS allowed for pitchers broken down by OBP and SLG components:

[16]:
```
sns.set(style='white')

ax = ops_df.loc[:,['PitcherId', 'OBP-allowed', 'SLG-allowed']].
 ↪set_index('PitcherId').plot(kind='bar', stacked=True, color=['gray', 'red'],␣
 ↪width = 1)

plt.title('OPS allowed by Pitcher (desc by Batters Faced)', fontsize=16)

plt.ylabel('OPS')

ax.bar_label(ax.containers[0], label_type='center', fmt = '{0:.2f}')
ax.bar_label(ax.containers[1], label_type='center', fmt = '{0:.2f}')
plt.axhline(y = .712, color = 'black', linestyle = 'dashed', label = 'MLB avg␣
 ↪OPS')
plt.legend(bbox_to_anchor = (1.0, 1), loc = 'upper center')
plt.xticks(rotation=0)
```

[16]:
```
(array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16]),
 [Text(0, 0, '1'),
  Text(1, 0, '7'),
  Text(2, 0, '11'),
  Text(3, 0, '10'),
  Text(4, 0, '6'),
  Text(5, 0, '12'),
  Text(6, 0, '14'),
  Text(7, 0, '3'),
  Text(8, 0, '5'),
  Text(9, 0, '17'),
  Text(10, 0, '9'),
  Text(11, 0, '4'),
  Text(12, 0, '2'),
  Text(13, 0, '8'),
  Text(14, 0, '13'),
  Text(15, 0, '15'),
  Text(16, 0, '16')])
```



- The above visual sorts pitchers by at-bats faced, and plots their OPS allowed (by OBP and SLG).
- OPS is a comprehensive stat that measures pitcher ability to limit baserunners and XBH, preventing runs. Pitchers want an OPS **lower than MLB avg of .713**.
- The top 3 pitchers in batters faced had a proficient OPS allowed, with **pitcher 7** showing the best performance : **.50 OPS allowed** through 26 batters.
- **Pitcher 6** was perfect throug 2 innings, and **pitcher 3** allowed only **.40 OPS** through 5 batters (.20 OBP and SLG).
- **Pitchers 10, 12, and 14** performed poorly, allowing high OPS numbers. **Pitcher 14** in particular allowed almost 1 base per batter **(.8 SLG)** for 5 batters, giving him the highest OPS allowed (worst).

1.6 K/9

[17]:
```
K_df = []
for pitcher in pitcher_atbats_calls2['PitcherId'].unique().tolist():
    p_data = pitcher_atbats_calls2[pitcher_atbats_calls2['PitcherId'] ==␣
 ↪pitcher]
    p_Ks = p_data[p_data['PitchCall'] == 'strikeout']['Count'].values.sum()
```

```
    IP = innings_pitched_final[innings_pitched_final['PitcherId'] ==␣
 ↪pitcher]['IP'].values[0]
    k_per_nine = p_Ks * (9/IP)
    K_df.append([pitcher, k_per_nine, IP])
K_df = pd.DataFrame(K_df, columns=['PitcherId', 'K/9', 'IP'])
K_df = K_df.sort_values(by='IP', ascending=False)
K_df
```

[17]:

	PitcherId	K/9	IP
0	1	9.750000	12
10	11	6.428571	7
6	7	7.500000	6
5	6	9.000000	2
2	3	13.500000	2
4	5	0.000000	1
3	4	0.000000	1
7	8	0.000000	1
1	2	18.000000	1
8	9	27.000000	1
9	10	0.000000	1
11	12	9.000000	1
12	13	0.000000	1
13	14	0.000000	1
14	15	18.000000	1
15	16	9.000000	1
16	17	0.000000	1

- Examining K/9 gives us more insight into how deceptive the pitchers' "stuff" is, and how that contributed to their performance.
- Evidently, **pitcher 1** utilized his strikeout ability **(9.7 K/9)** to reach his impressive **.60 OPS** allowed through the most innings appeared in (12).
- **Pitcher 7**, the best performer by OPS allowed, didn't rely on strikeouts to get outs although **7.5 K/9** is solid.
- **Pitcher 11** the next most productive pitcher, averaged **less than 1 strikeout per inning** appeared in.
- Our susceptible pitchers such as **10 and 14** didn't manage a strikeout through their 1 inning
- **Pitchers 3 and 6** who were wildly successful in their relief outings, showed high strikeout ability **(13.5 K/9 and 9 K/9)**

1.7 K-BB%

[18]:
```
K_BB_df = []
for pitcher in pitcher_atbats_calls2['PitcherId'].unique().tolist():
    p_data = pitcher_atbats_calls2[pitcher_atbats_calls2['PitcherId'] ==␣
 ↪pitcher]
```

```
    p_Ks = p_data[p_data['PitchCall'] == 'strikeout']['Count'].values.sum()
    p_BBs = p_data[p_data['PitchCall'] == 'walk']['Count'].values.sum()
    ABs = len(p_data['AtBatNumber'].unique())
    k_rate = (p_Ks / ABs) * 100
    bb_rate = (p_BBs / ABs) * 100
    K_minus_BB = k_rate - bb_rate
    K_BB_df.append([pitcher, k_rate, bb_rate, K_minus_BB, ABs])
K_BB_df = pd.DataFrame(K_BB_df, columns=['PitcherId', 'K%', 'BB%', 'K-BB%',␣
 ↪'At-Bats'])
K_BB_df = K_BB_df.sort_values(by='At-Bats', ascending=False)
K_BB_df
```

[18]:

	PitcherId	K%	BB%	K-BB%	At-Bats
0	1	33.333333	7.692308	25.641026	39
6	7	19.230769	7.692308	11.538462	26
10	11	19.230769	3.846154	15.384615	26
9	10	0.000000	16.666667	-16.666667	6
5	6	33.333333	0.000000	33.333333	6
11	12	16.666667	0.000000	16.666667	6
13	14	0.000000	20.000000	-20.000000	5
2	3	60.000000	0.000000	60.000000	5
4	5	0.000000	0.000000	0.000000	5
16	17	0.000000	20.000000	-20.000000	5
8	9	75.000000	0.000000	75.000000	4
3	4	0.000000	25.000000	-25.000000	4
1	2	66.666667	0.000000	66.666667	3
7	8	0.000000	33.333333	-33.333333	3
12	13	0.000000	0.000000	0.000000	3
14	15	66.666667	0.000000	66.666667	3
15	16	50.000000	0.000000	50.000000	2

[19]:

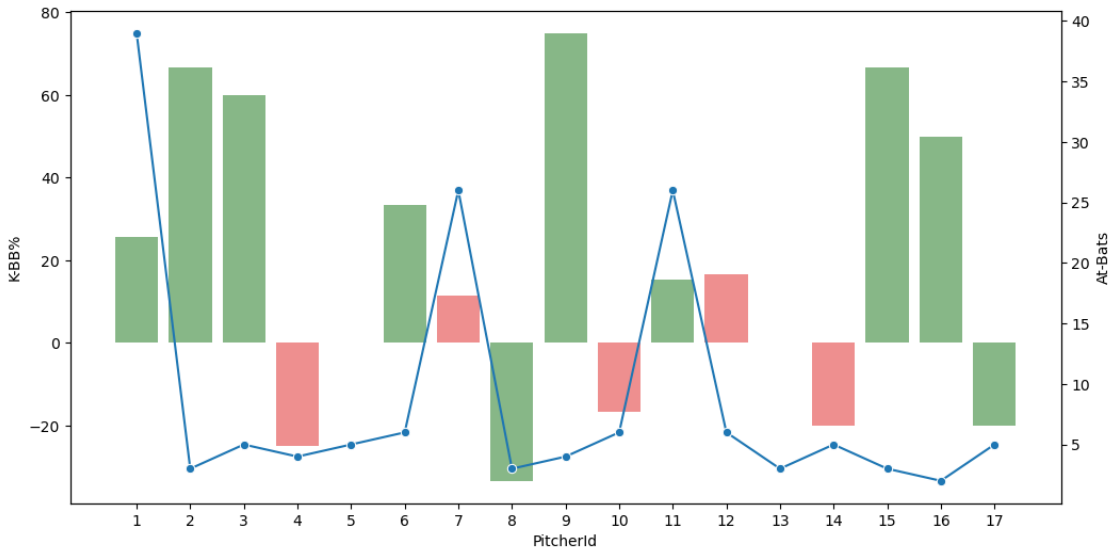
```
import matplotlib
matplotlib.rc_file_defaults()
ax1 = sns.set_style(style=None, rc=None )

fig, ax1 = plt.subplots(figsize=(12,6))

ax2 = ax1.twinx()
cols = ['red' if y < 0 else 'green' for y in K_BB_df['K-BB%']]
sns.barplot(data = K_BB_df, x='PitcherId', y='K-BB%', alpha=0.5, ax=ax1,␣
 ↪errorbar=('ci', 95), palette=cols)
sns.lineplot(data = K_BB_df['At-Bats'], marker='o', sort = True, ax=ax2)
```

```
/tmp/ipykernel_114/2362082186.py:9: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
```

effect.

```
  sns.barplot(data = K_BB_df, x='PitcherId', y='K-BB%', alpha=0.5, ax=ax1,
errorbar=('ci', 95), palette=cols)
```

[19]: <Axes: ylabel='At-Bats'>



- The above graph shows K-BB% for each Pitcher, with the Number of At-Bats delineated by the line and the right axis
- **Pitcher 1** employed his ability to strikeout hitters without walking them (**26% K-BB%**), demonstrating control and deceptive stuff
- **Pitcher 7** was most effective as we saw in the OPS analysis, but didn't rely on the strikeout **(11% K-BB%)**, indicating that his arsenal wasn't overpowering but he controlled the strike zone
- Pitchers that failed to record strikeouts, but walked batters appear as negative bars

1.8 7.3 Underlying Pitch-Level Pitching KPIs

1.8.1 KPIs of interest: Whiff%, Strike/Putout%

1.8.2 Whiff %

[20]:
```
pitch_thrown_df = pitchdata.query("PitchCall != 'pickoff_attempt_1b'")
swingingstrikes = pitch_thrown_df.query("PitchCall == 'swinging_strike' or␣
↪PitchCall == 'swinging_strike_blocked'")
swingingstrikes = swingingstrikes.groupby('PitcherId').size().to_frame().
↪reset_index()
swingingstrikes = swingingstrikes.rename({0:'Num Whiffs'}, axis=1)
```

```
numpitches = pitch_thrown_df.groupby('PitcherId').size().
↪sort_values(ascending=False).to_frame().reset_index()
numpitches = numpitches.rename({0:'Pitches Thrown'}, axis=1)
merged_whiff_stats = pd.merge(numpitches, swingingstrikes, on='PitcherId', how␣
↪='left')
merged_whiff_stats['Num Whiffs'] = merged_whiff_stats['Num Whiffs'].fillna(0)
```

[21]:
```
merged_whiff_stats['Whiff_%'] = merged_whiff_stats['Num Whiffs'] /␣
↪merged_whiff_stats['Pitches Thrown'] *100
merged_whiff_stats
```

[21]:

	PitcherId	Pitches Thrown	Num Whiffs	Whiff_%
0	1	198	14.0	7.070707
1	11	97	5.0	5.154639
2	7	75	6.0	8.000000
3	5	26	3.0	11.538462
4	6	23	0.0	0.000000
5	9	22	1.0	4.545455
6	3	21	1.0	4.761905
7	12	20	3.0	15.000000
8	10	17	1.0	5.882353
9	15	17	2.0	11.764706
10	13	15	2.0	13.333333
11	17	15	0.0	0.000000
12	14	14	0.0	0.000000
13	4	12	0.0	0.000000
14	16	11	1.0	9.090909
15	2	10	0.0	0.000000
16	8	8	0.0	0.000000

1.8.3 Strike/PutOut %

[22]:
```
strikes_or_putouts = pitch_thrown_df[pitch_thrown_df['PitchCall'].
↪isin(strike_or_putout)]
strikes_or_putouts = strikes_or_putouts.groupby('PitcherId').size().to_frame().
↪reset_index()
strikes_or_putouts= strikes_or_putouts.rename({0:'Num Strikes or Putouts'},␣
↪axis=1)
merged_strikes_stats = pd.merge(numpitches, strikes_or_putouts, on='PitcherId',␣
↪how ='left')
merged_strikes_stats['Num Strikes or Putouts'] = merged_strikes_stats['Num␣
↪Strikes or Putouts'].fillna(0)
merged_strikes_stats['Strikes/putout_%'] = merged_strikes_stats['Num Strikes or␣
↪Putouts'] / merged_strikes_stats['Pitches Thrown'] *100
merged_strikes_stats
```

[22]:

	PitcherId	Pitches Thrown	Num Strikes or Putouts	Strikes/putout_%
0	1	198	124	62.626263
1	11	97	55	56.701031
2	7	75	42	56.000000
3	5	26	14	53.846154
4	6	23	16	69.565217
5	9	22	14	63.636364
6	3	21	11	52.380952
7	12	20	11	55.000000
8	10	17	7	41.176471
9	15	17	10	58.823529
10	13	15	7	46.666667
11	17	15	6	40.000000
12	14	14	6	42.857143
13	4	12	6	50.000000
14	16	11	5	45.454545
15	2	10	8	80.000000
16	8	8	1	12.500000

1.9 7.4 Pitch-Type KPIs for Pitcher 1

1.9.1 KPIs of interest: Usage%, Strike%:

[23]:
```
#group by pitcher, pitchtype, pitch call
most_abs = [1, 7, 11]
pitcher_pitchtype_calls = pitchdata.groupby(['PitcherId', 'PitchType',␣
↪'PitchCall']).size().to_frame().reset_index()
pitcher_pitchtypes_calls2 = pitcher_pitchtype_calls.rename({0:'Count'}, axis =␣
↪1)
ppc3 = pitcher_pitchtypes_calls2[pitcher_pitchtypes_calls2['PitcherId'].
↪isin(most_abs)]
```

[25]:
```
full_pitchers_df = []
for pitcher in most_abs:
    p_data = ppc3[ppc3['PitcherId'] == pitcher]
    num_pitches_each = p_data.groupby('PitchType')['Count'].agg('sum').
↪to_frame().reset_index()
    num_pitches_each = num_pitches_each.rename({'Count':'Pitch Count'}, axis =␣
↪1)
    strikes_or_putouts = p_data[p_data['PitchCall'].isin(strike_or_putout)]
    strikes_or_putouts = strikes_or_putouts.groupby('PitchType')['Count'].
↪agg('sum').to_frame().reset_index()
    strikes_or_putouts = strikes_or_putouts.rename({'Count':'Num Strikes or␣
↪Putouts'}, axis=1)
    merged_pitchtype_stats = pd.merge(num_pitches_each, strikes_or_putouts,␣
↪on='PitchType', how ='left')
    full_pitchers_df.append(merged_pitchtype_stats)
```

[26]:
```
pitcher_1 = full_pitchers_df[0]
total_pc = pitcher_1['Pitch Count'].values.sum()
pitcher_1['Strikes/putout_%'] = pitcher_1['Num Strikes or Putouts'] /␣
↪pitcher_1['Pitch Count'] * 100
pitcher_1['usage %'] = pitcher_1['Pitch Count'] / total_pc * 100
```

[28]:
```
pitcher_1 = pitcher_1.loc[:, ['PitchType', 'Strikes/putout_%', 'usage %']]
```

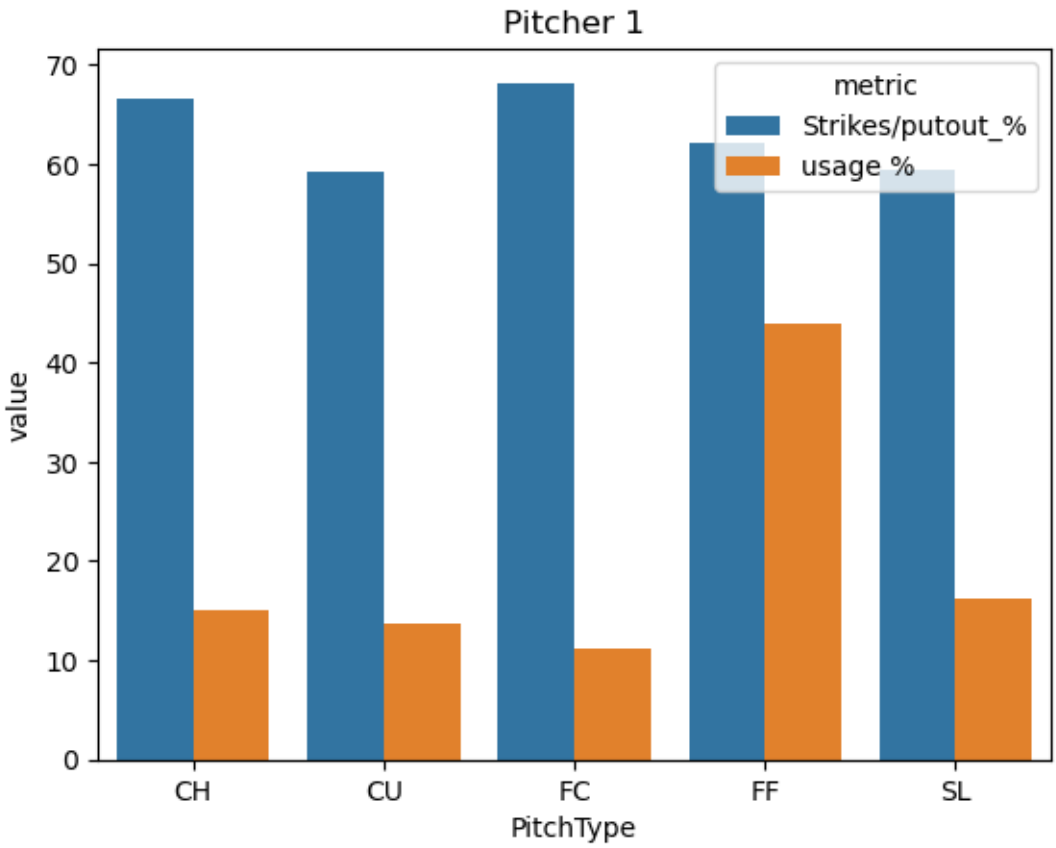
1.9.2 Pitcher 1 Usage and Efficiency by Pitch Type

[29]:
```
pitcher_1
```

[29]:

	PitchType	Strikes/putout_%	usage %
0	CH	66.666667	15.151515
1	CU	59.259259	13.636364
2	FC	68.181818	11.111111
3	FF	62.068966	43.939394
4	SL	59.375000	16.161616

[45]:
```
pitcher_1_df = pitcher_1.melt(id_vars=["PitchType"], var_name="metric")
sns.barplot(x = 'PitchType', y = 'value', hue='metric', data = pitcher_1_df)
plt.title('Pitcher 1')
# Show the plot
plt.show()
```



- Having the most pitches thrown among the two games, we investigate pitcher 1 performance on the pitch-type level:
- He relied on his **Four-Seam and Slider** interplay, accounting for **60% of his total usage**. each produced about a **strike or putout about 60-70%** of the time, accounting for his productive **sub-1.0 WHIP, elite K-BB%, and efficient .60 OPS**

1.10 7.4 Tracking KPIs for Pitcher 1

1.10.1 KPIs of interest: Speed, Spin Rate, Hor. Break, Vert. Break, Spin Fit Error

1.10.2 Pitcher 1 Pitch Tracking Table

[39]:
```
p1 = pitchdata[pitchdata['PitcherId'] ==1]
p1.groupby('PitchType')[['ReleaseSpeed', 'ReleaseSpinRate',␣
↪'TrajectoryHorizontalBreak', 'TrajectoryVerticalBreakInduced',␣
↪'SpinFitError']].agg('mean')
```

[39]:
```
            ReleaseSpeed  ReleaseSpinRate  TrajectoryHorizontalBreak  \
PitchType
CH             86.132221      1586.087435                   1.122591
CU             77.174687      2640.545085                  -1.039133
```

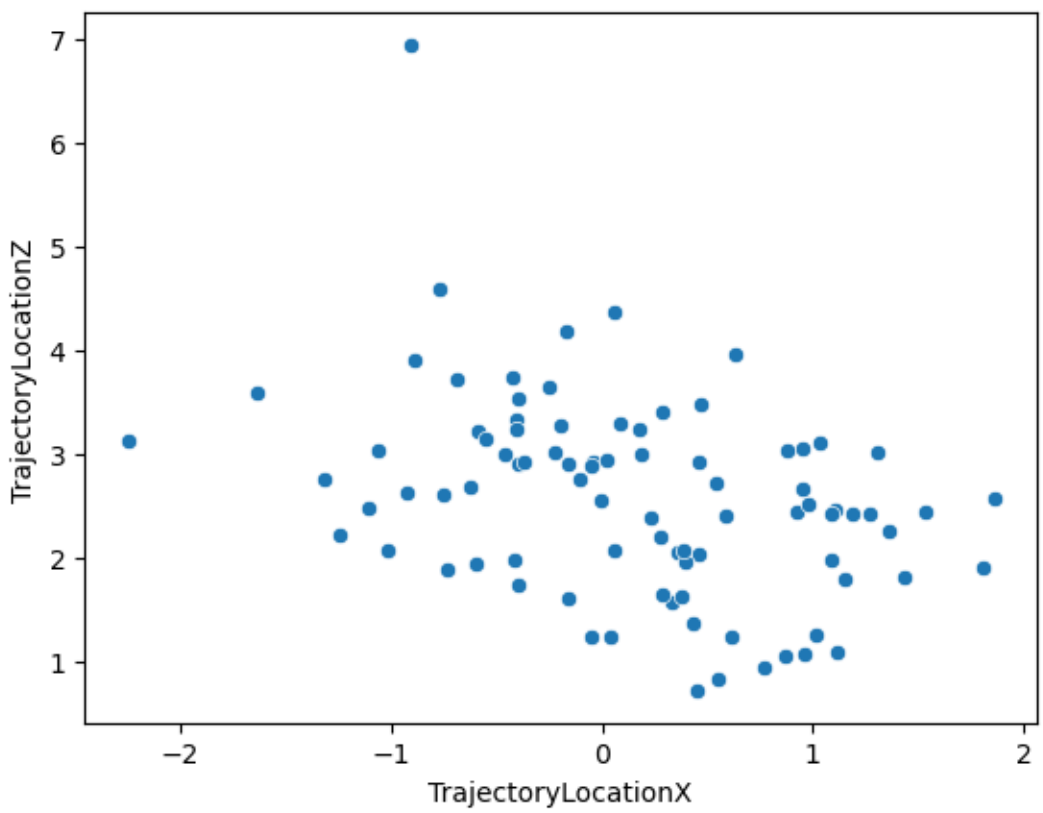
```
FC            91.645463      2425.782815                   0.103738
FF            94.822126      2391.307786                   0.839050
SL            87.098559      2199.774986                  -0.246309

           TrajectoryVerticalBreakInduced  SpinFitError
PitchType
CH                               0.267953      0.178569
CU                              -0.678539      0.187410
FC                               0.731695      0.186740
FF                               1.156294      0.166756
SL                               0.444775      0.162926
```

- Average value of pitch tracking metrics allow us to identify aspects such as the impressive induced vertical break on pitcher 1's fastball

1.10.3 Locations of Pitcher 1 Four Seam

[40]:
```
p1_FF = p1[p1['PitchType']=='FF']
sns.scatterplot(x='TrajectoryLocationX', y='TrajectoryLocationZ', data=p1_FF)
```

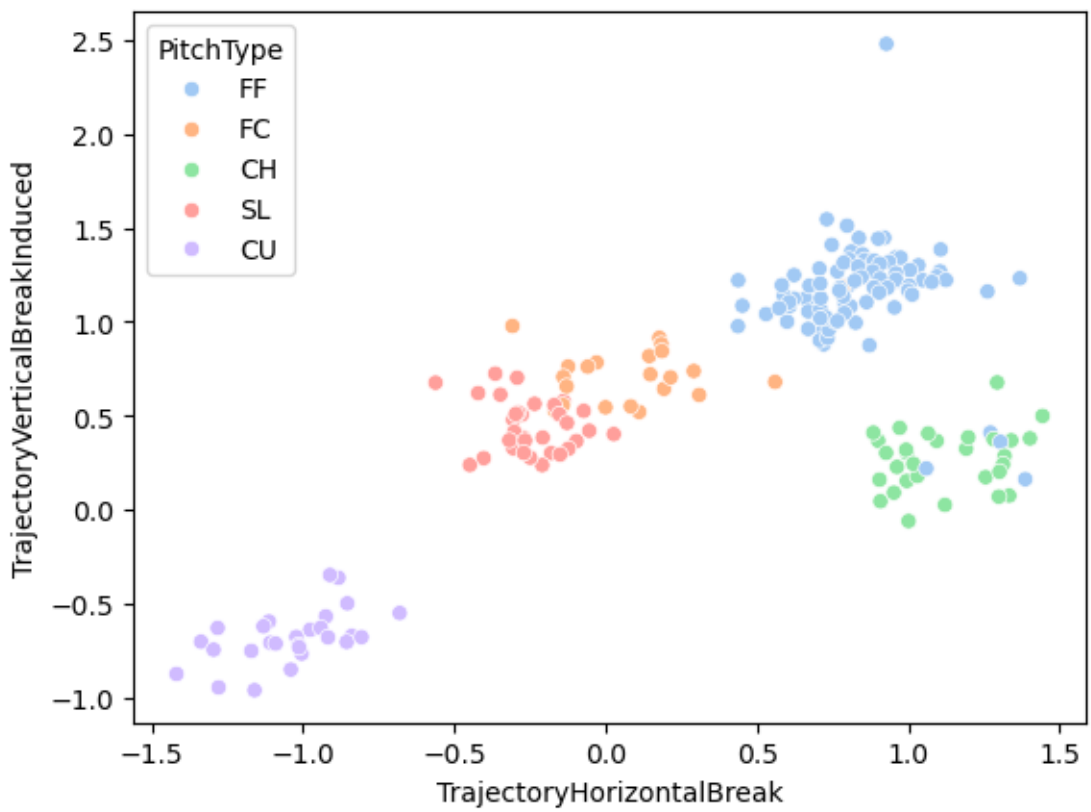
[40]: <Axes: xlabel='TrajectoryLocationX', ylabel='TrajectoryLocationZ'>



1.10.4 Pitcher 1 Pitch Movement by Pitch Type

[43]:
```
sns.scatterplot(x='TrajectoryHorizontalBreak',␣
↪y='TrajectoryVerticalBreakInduced', data=p1, hue='PitchType',␣
↪palette='pastel')
```

[43]:
```
<Axes: xlabel='TrajectoryHorizontalBreak',
ylabel='TrajectoryVerticalBreakInduced'>
```



-The Four-Seam and Slider combo is illustrated here by pitch movement. While the Four-Seam breaks toward the top right of the zone, the Slider breaks the opposite way.